

LDPC codes

Based on MSA,SPA Algorithm



潘俊宏



Outline

- 1.Introduction
- 2.Block Diagram
- 3.Finite State Machine
- 4.Code Description
- 5.Problem
- 6.Simulation Data
- 7.Simulation Figure

Introduction

- Consider the (1023,781) LDPC code with block length $n=1023$, dimension $k=781$, row weight $\rho = 32$ and column weight $\gamma = 32$
- Adopt Soft-Decision Decoding (SPA(alternative method , MSA)

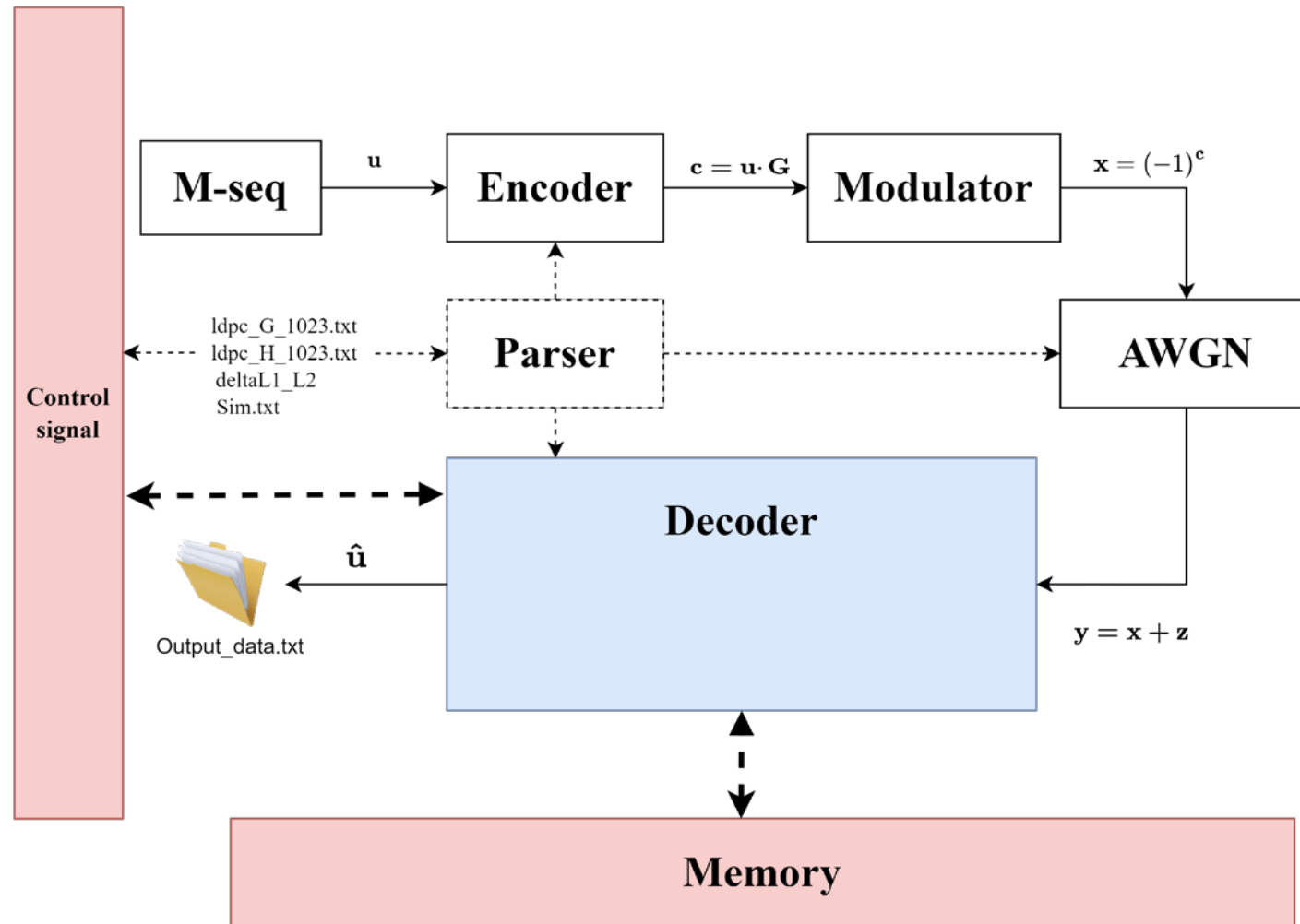
- SPA(sum-product) $\text{VAR}(L_1, L_2) = \ln(\lambda_1 \lambda_2) = \ln(\lambda_1) + \ln(\lambda_2) = L_1 + L_2$

$$\text{CHK}(L_1, L_2) = 2 \tanh^{-1} \left(\tanh \left(\frac{L_1}{2} \right) \tanh \left(\frac{L_2}{2} \right) \right)$$

- MSA(min-sum) $\text{VAR}(L_1, L_2) = \ln(\lambda_1 \lambda_2) = \ln(\lambda_1) + \ln(\lambda_2) = L_1 + L_2$
 $\text{CHK}(L_1, L_2) = \text{sgn}(L_1) \text{sgn}(L_2) \min(|L_1|, |L_2|)$

- SPA(alternative) $\text{VAR}(L_1, L_2) = \ln(\lambda_1 \lambda_2) = \ln(\lambda_1) + \ln(\lambda_2) = L_1 + L_2$
 $\text{CHK}(L_1, L_2) = \text{sgn}(L_1) \text{sgn}(L_2) \min(|L_1|, |L_2|) + \Delta(L_1, L_2)$

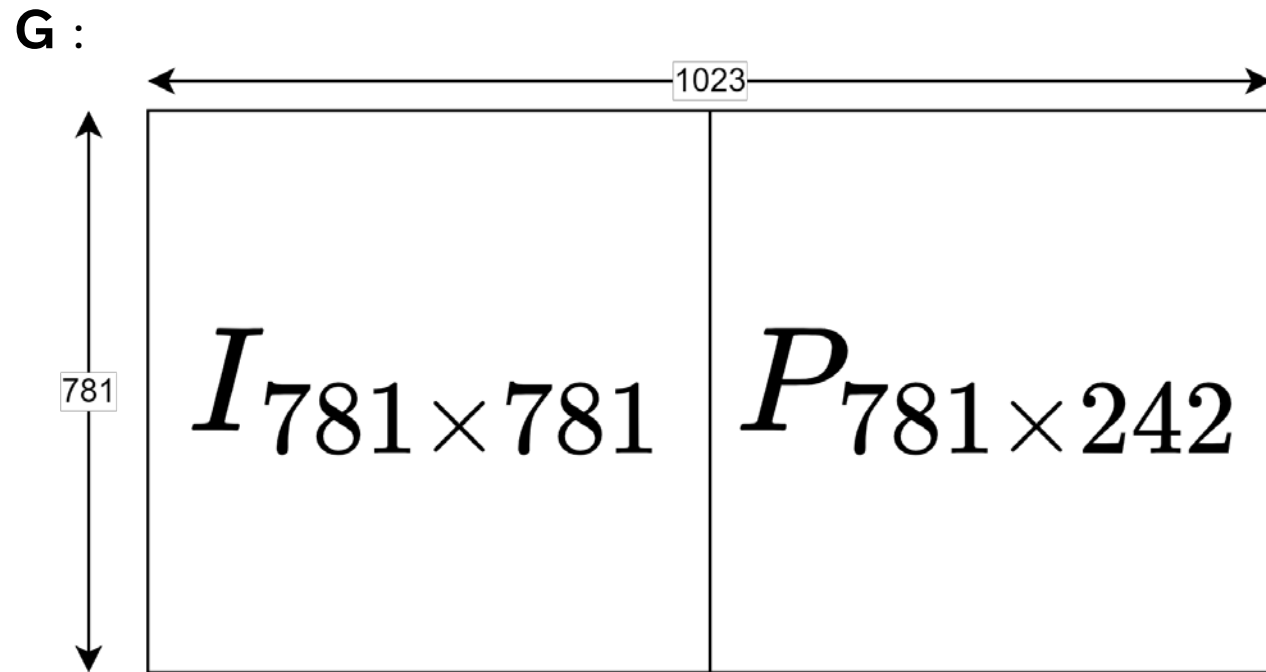
Block Diagram



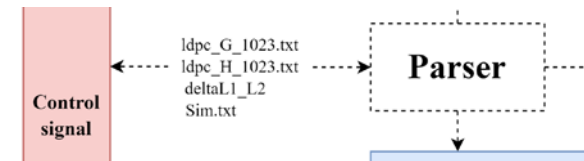
- Adopt Soft-Decision Decoding
- $\mathbf{u} = [u_0 \ u_1 \ \cdots \ u_{780}]^T$, where $\mathbf{u}$ is message
- $\mathbf{c} = [c_0 \ c_1 \ \cdots \ c_{1022}]^T$, where $\mathbf{c}$ is codeword
- $\mathbf{x} = [x_0 \ x_1 \ \cdots \ x_{1022}]^T$, where $\mathbf{c}$ is BPSK signal
- $\mathbf{y} = [y_0 \ y_1 \ \cdots \ y_{1022}]^T$, where $\mathbf{y}$ is signal Interfered by noise
- $\hat{\mathbf{u}} = [\hat{u}_0 \ \hat{u}_1 \ \cdots \ \hat{u}_{780}]^T$, where $\hat{u}_0$ is decoded message
- $\mathbf{G} = [\mathbf{g}_0 \ \mathbf{g}_1 \ \cdots \ \mathbf{g}_{1022}]^T$, where $\mathbf{g}$ is codeword basis

Block Diagram

ldpc_G_1023 :

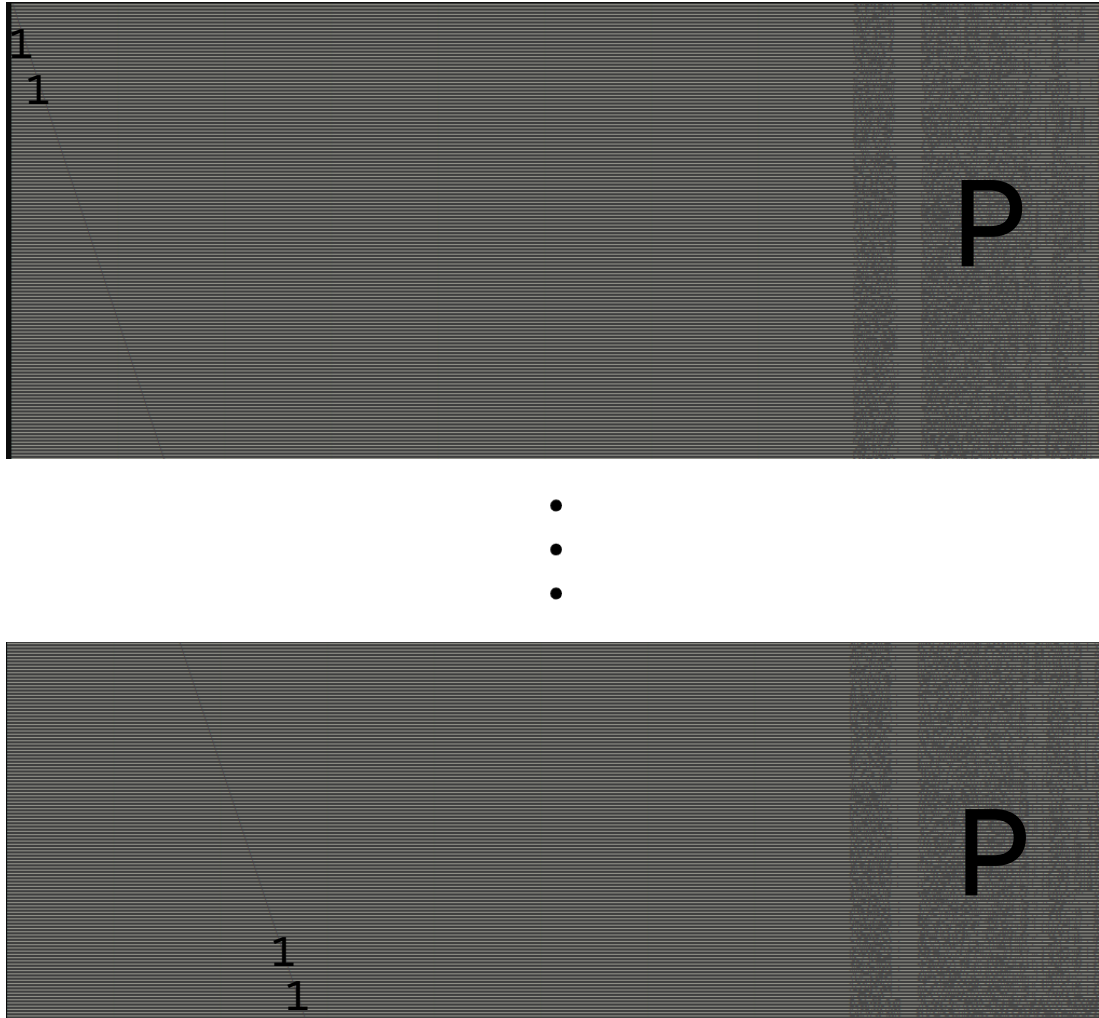


- Adopt Soft-Decision Decoding
- $\mathbf{u} = [u_0 \ u_1 \ \cdots \ u_{780}]^T$, where $\mathbf{u}$ is message
- $\mathbf{c} = [c_0 \ c_1 \ \cdots \ c_{1022}]^T$, where $\mathbf{c}$ is codeword
- $\mathbf{x} = [x_0 \ x_1 \ \cdots \ x_{1022}]^T$, where $\mathbf{c}$ is BPSK signal
- $\mathbf{y} = [y_0 \ y_1 \ \cdots \ y_{1022}]^T$, where $\mathbf{y}$ is signal Interfered by noise
- $\hat{\mathbf{u}} = [\hat{u}_0 \ \hat{u}_1 \ \cdots \ \hat{u}_{780}]^T$, where $\hat{\mathbf{u}}$ is decoded message
- $\mathbf{G} = [\mathbf{g}_0 \ \mathbf{g}_1 \ \cdots \ \mathbf{g}_{1022}]^T$, where $\mathbf{g}$ is codeword basis

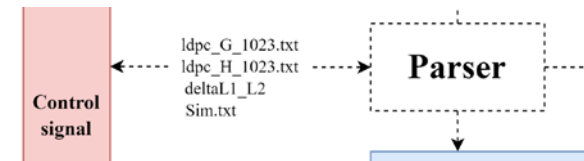


Block Diagram

ldpc_G_1023 :



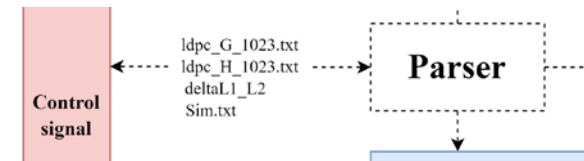
- Adopt Soft-Decision Decoding
- $\mathbf{u} = [u_0 \ u_1 \ \cdots \ u_{780}]^T$, where $\mathbf{u}$ is message
- $\mathbf{c} = [c_0 \ c_1 \ \cdots \ c_{1022}]^T$, where $\mathbf{c}$ is codeword
- $\mathbf{x} = [x_0 \ x_1 \ \cdots \ x_{1022}]^T$, where $\mathbf{c}$ is BPSK signal
- $\mathbf{y} = [y_0 \ y_1 \ \cdots \ y_{1022}]^T$, where $\mathbf{y}$ is signal Interfered by noise
- $\hat{\mathbf{u}} = [\hat{u}_0 \ \hat{u}_1 \ \cdots \ \hat{u}_{780}]^T$, where $\hat{u}_0$ is decoded message
- $\mathbf{G} = [\mathbf{g}_0 \ \mathbf{g}_1 \ \cdots \ \mathbf{g}_{1022}]^T$, where $\mathbf{g}$ is codeword basis



Block Diagram

- Sim.txt

```
1023    %N: length of codeword
781     %M: length of message
50      %E_B: number of error block
150     %Ite : max number of iteration
32      %Gr:number of 1's in every column
32      %Rho  :umber of 1's in every row
0.0     %SNR: bit signal-to-noise ratio (dB)
-2023   %Seed: a negative integer
0       %0=>SPA; 1=>MSA ;
```



Block Diagram

- Output_data.txt

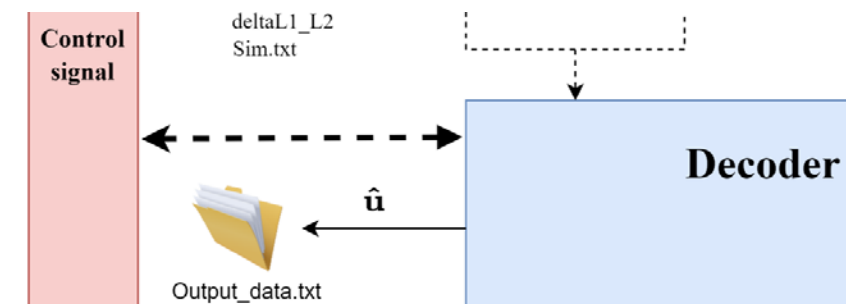


```
% SNR= 0.000000, max error_B= 50, max_ite= 50
% the arguments are
(Block accumulated ,Error block accumulated, No.Error bit, No.Iteration) ,repectively
1 1 86 50
2 2 87 50
3 3 73 50
4 4 80 50
```

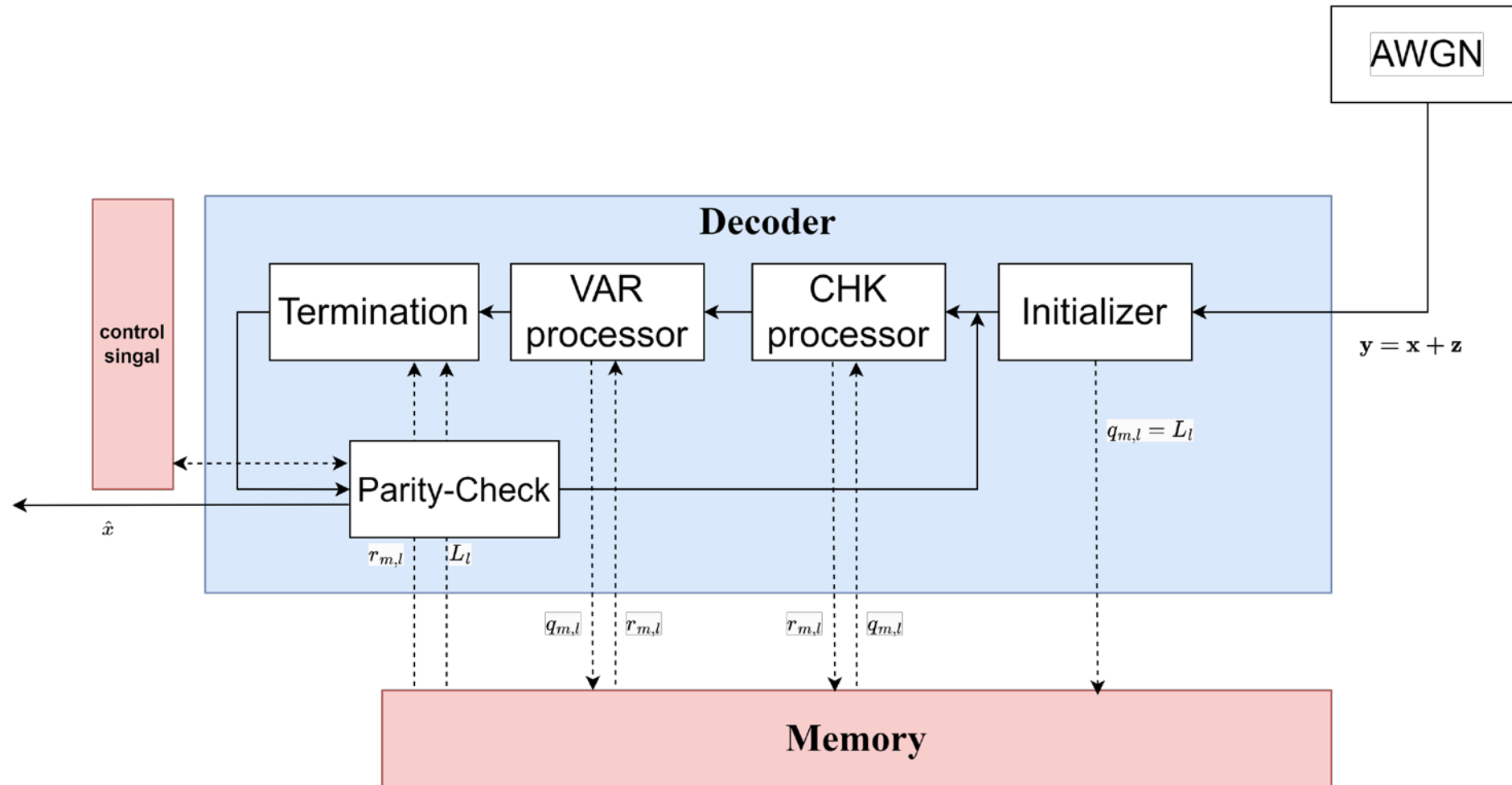
•
•
•

```
48 48 71 50
49 49 72 50
50 50 85 50

time cost= 81.698000
% (Total decoded bit, Total error bit, BER, BLER)
39050 4251 1.088604E-001 1.000000E+000
```



Block Diagram

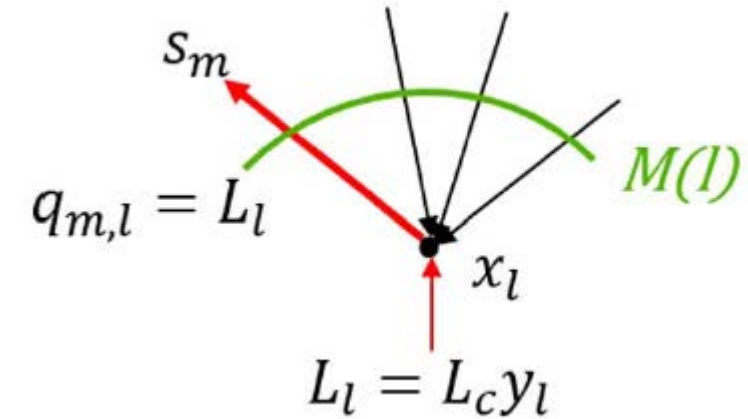
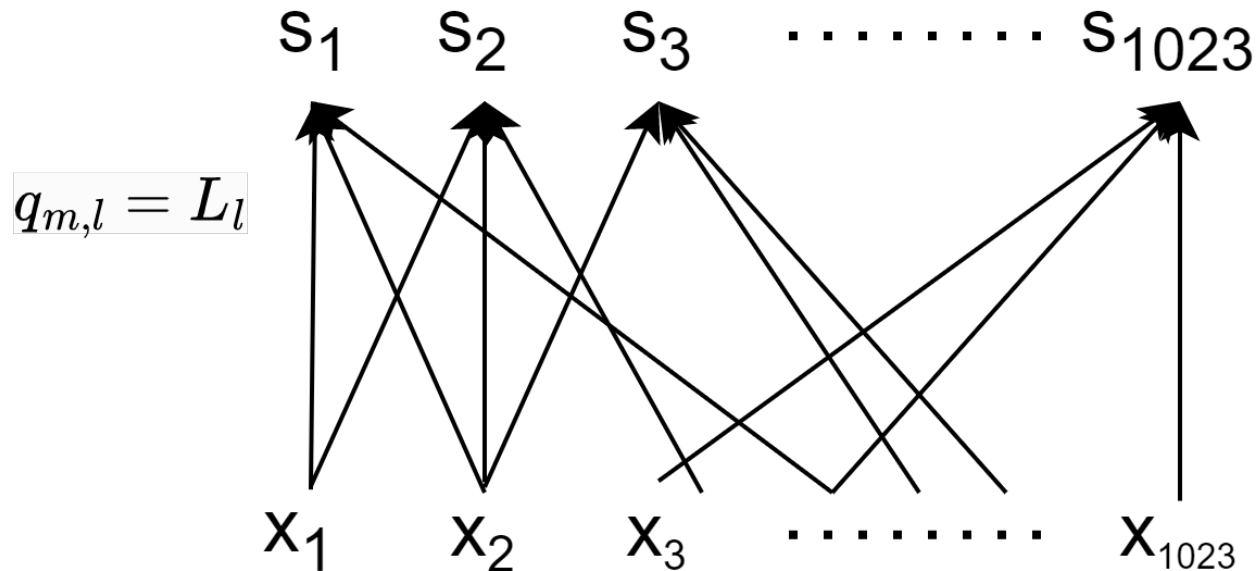


Block Diagram

- Initializer

where L_c is $4R \frac{E_b}{N_0} = 4 \frac{E_s}{N_0}$

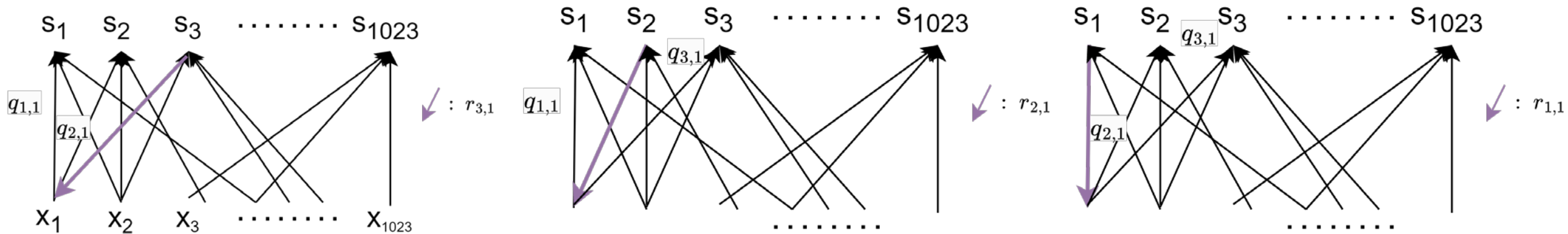
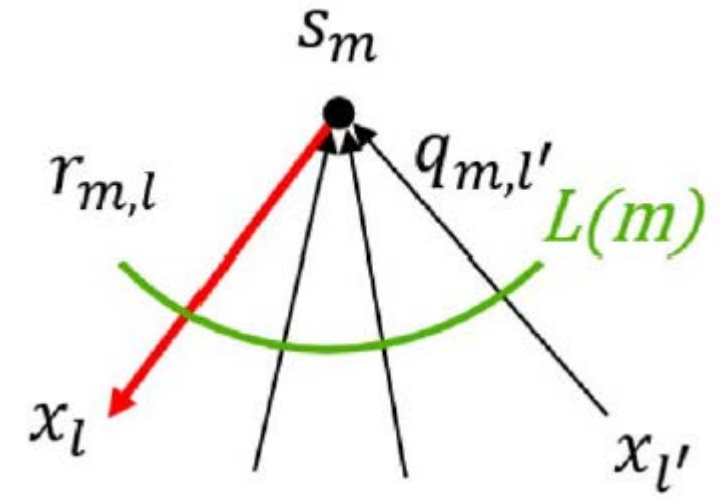
the very first $q_{m,l} = L_c \cdot y$



Block Diagram

- Check Operation

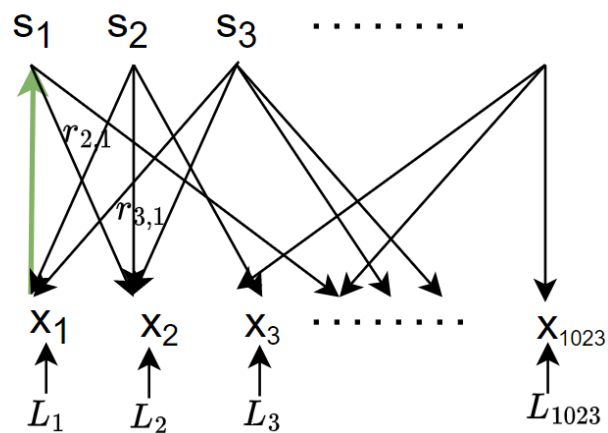
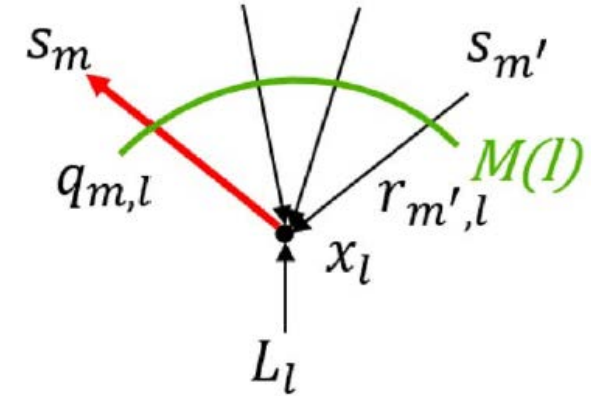
$$\text{CHK}(x_1, x_2, \dots, x_n) = \text{CHK}(x_1, \text{CHK}(x_2, \dots, x_n)).$$



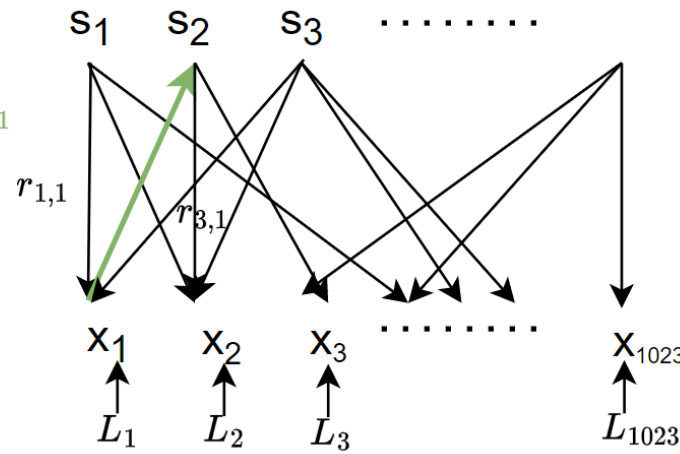
Block Diagram

- VAR Operation

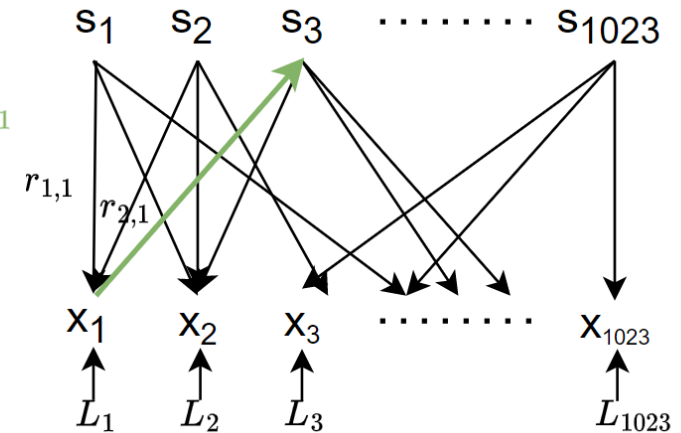
$$\text{VAR}(x_1, x_2, \dots, x_n) = \text{VAR}(x_1, \text{VAR}(x_2, \dots, x_n))$$



↑ : $q_{1,1}$



↑ : $q_{2,1}$



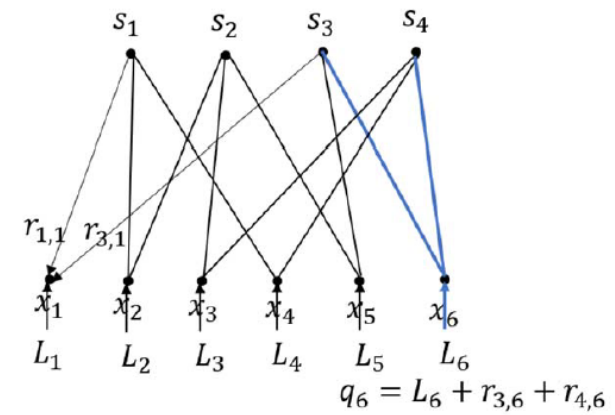
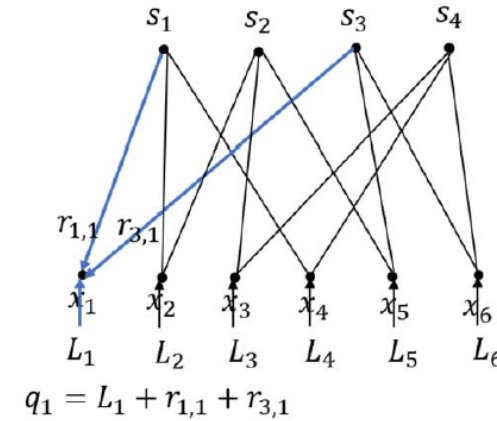
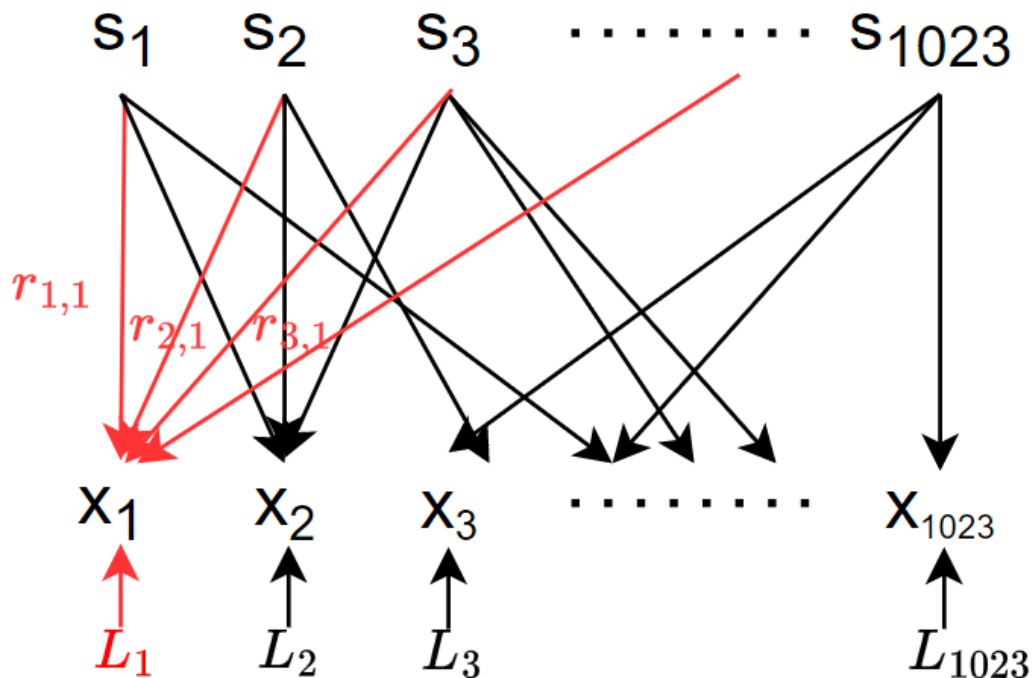
↑ : $q_{3,1}$

Block Diagram

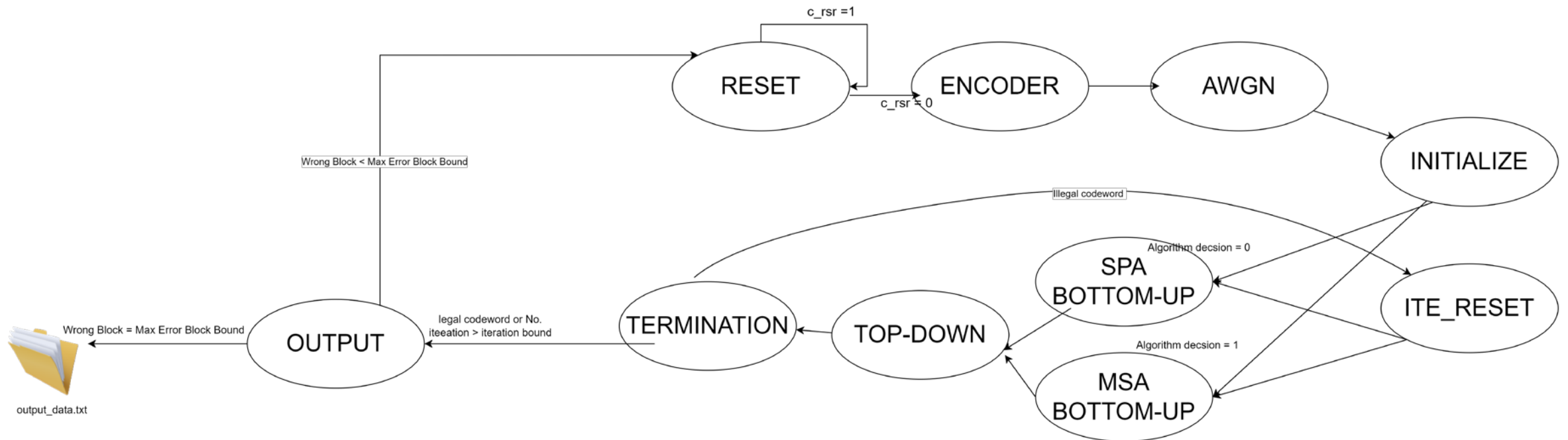
- Termination Operation

For $1 \leq l \leq n$,

$$\hat{x}_l = \begin{cases} 0, & \text{if } q_l > 0; \\ 1, & \text{if } q_l < 0. \end{cases}$$



Finite State Machine





Code Description

```
while(enable){ // while control signal "enable is on, starting the FSM"
    if(rst) state = RESET; //the very first condition, control signal "rst" is on
    else state = next_state;
    switch(state){

    case RESET:
        All value to 0
        next_state = ENC ;
        break ;
```

Code Description

```
case ENC:
    if(No. accumulated decoded block==0) Message[0] = 1;
    else{ // initial state
        for(i=0;i<6;i++){
            Message[i] = Initial_state[i]^Initial_state[i+1];
        }
    }
    M_sequence function (length of Message,initial state,) // m-sequence generating

    for(i=0;i<M;i++){
        if(Message[i]==0) go back to for llop
        else{
            Codeword[i] = 1;
            for(j=M;j<N;j++){
                Codeword[j] ^= GEn[i][j] ; // generate codeword from ldpc_G_1023.txt file
            }
        }
    }
    for(i=0;i<N;i++){Codeword_map_noise[i] = (Codeword[i])? -1:1 ;} // BPSK
    next_state = AWGN ;
    break; // go to next state
```

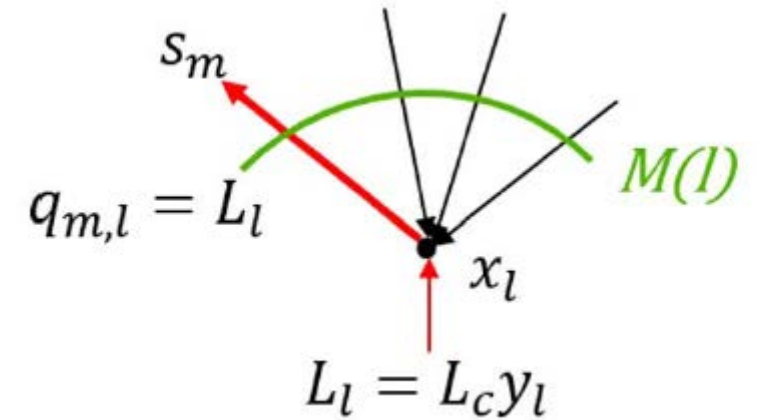
Code Description

```
case AWGN:
    for (i=0;i<512;i++){
        random_number1=random_function(Seed_pointer); //every output is different value
        random_number1=random_function(Seed_pointer);
        if(i==511){
            Codeword_map_noise[i+i] = Codeword_map_noise[i+i] + *random_number1;
        }
        else{
            Codeword_map_noise[i+i] = Codeword_map_noise[i+i] + *random_number1;
            Codeword_map_noise[i+i+1] = Codeword_map_noise[i+i+1] + *random_number2;
        }
    } //for end
    next_state = INITIALIZE ;
    break ;
```

Code Description

case INITIALIZE:

```
Lc = 4*R*SNR_unit;  
for(i=0;i<N;i++){  
    f_temp = Lc * Codeword_map_noise[i];  
    for(j=0;j<Gr;j++){  
        LLR_qml[i][j] = f_temp ;  
    }  
    INI_Ll[i] = f_temp ;  
}  
  
if(Algorithm_decision==1) next_state = BOTTOM_UP_SPA ;  
else if(Algorithm_decision==0) next_state = BOTTOM_UP_MSA ;  
break;
```



Code Description

```

case BOTTOM_UP_SPA:
    for(i=0;i<N;i++){ //i is ith s
        for(j=0;j<Rho;j++){ // j is jth S2V
            (index, index2) = ldpc_H_1023.txt[i][j] //get the index from ldpc_H_1023.txt
            Candidate[j] = LLR_qml[index][index2];
        } //j end
        for(j=0;j<Rho;j++){ // j is jth rml
            count = 0 ;
            f_temp2 = Candidate[(j+1)%Rho];
            while(count<Rho){
                if(count!=j && count!=((j+1)%Rho)){
                    if(fabs(f_temp2+Candidate[count])>1000) index = 999;
                    else index = (int)((fabs( f_temp2+Candidate[count]) )*100);
                    if(fabs(f_temp2-Candidate[count])>1000) index2 = 999;
                    else index2 = (int)((fabs( f_temp2-Candidate[count]) )*100);
                    if(index>=999 && index2>=999 ) f_temp = 0;
                    else if(index>=999 && index2<999) f_temp = DELta[index2][999];
                    else if(index<999 && index2>=999) f_temp = DELta[999][index];
                    else f_temp = DELta[index2][index];
                    c_choose_sgn = (signbit(f_temp2)==signbit(Candidate[count]))? 1:0 ;
                    switch(c_choose_sgn){
                        //1 && -1 will equal 1, it is not allow my mind , so I fix it as 1 == -1

```

- MSA is different from the term if there is delta function or not

get the delta term
from table



Code Description

case BOTTOM_UP_SPA:

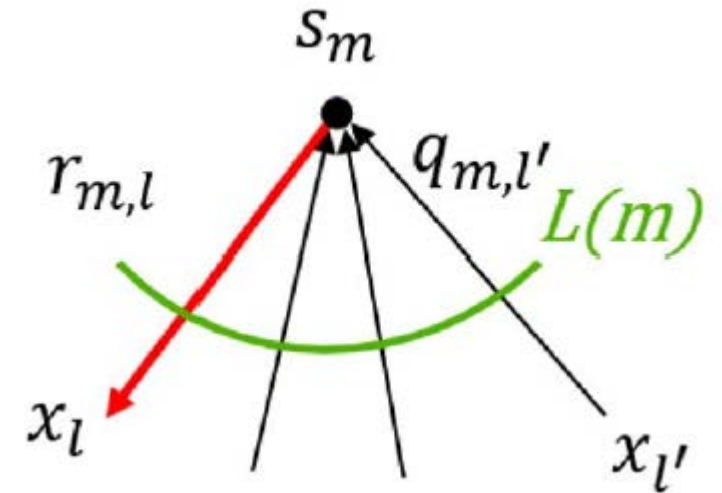
$$\text{CHK}(L_1, L_2) = \text{sgn}(L_1) \text{sgn}(L_2) \min(|L_1|, |L_2|) + \Delta(L_1, L_2)$$

case 1:

```
f_temp3 = D_abs(f_temp2) ;
f_temp4 = D_abs(Candidate[count]);
f_temp2 = D_min(f_temp3, f_temp4) + f_temp;
break;
```

case 0:

```
f_temp3 = D_abs(f_temp2) ;
f_temp4 = D_abs(Candidate[count]);
f_temp2 = -D_min(f_temp3, f_temp4) + f_temp;
break;
```



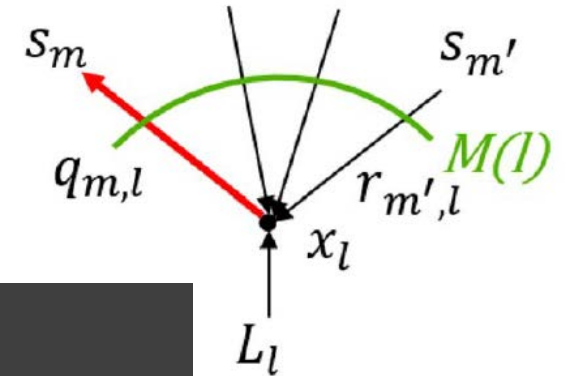
Code Description

case TOP_DOWN:

```

case TOP_DOWN:
    for(i=0;i<N;i++){ //i is ith s
        for(j=0;j<Rho;j++){ // j is jth S2V
            (index, index2) = ldpc_H_1023.txt[i][j] //get the index from ldpc_H_1023.txt
            Candidate[j] = LLR_rml[index][index2];
            f_temp2 += Candidate[j];
        }//j end
        for(j=0;j<32;j++){ // j is jth qml
            LLR_qml[i][j] = f_temp2 - Candidate[j]+INI_Ll[i];
        }//j end
        Survivor[i] = ((f_temp2+INI_Ll[i])>0)? 0:1 ; //Survivor[i] is my x hat
    }//for i end
    next_state = TERMINATION;
    break;

```





Code Description

case TERMINATION:

```
case TERMINATION:
    i_temp = N-M ;
    for(i=0;i<N;i++){
        if(Survivor[i] is valid codeword)
            control signal c_error is 0
        else control signal c_error is 1
    }
    if(c_error==0) next_state = OUTPUT ;
    else if (c_error && count_ite==Ite) next_state = OUTPUT;
    else next_state = ITE_RESET ;
    break;
```

Code Description

case OUTPUT:

- Check if my No. iteration has been to iteration bound or not
- Count my No. error block, total error bit and total block

```
case OUTPUT:
    count_allblk += 1;
    if(c_error && count_ite==Ite) {
        count_errorblk += 1 ;
        for(i=0;i<M;i++){
            if((Survivor[i]!=Message[i])) count_errorbit++;
        }
        sum += count_errorbit ;
    }
```

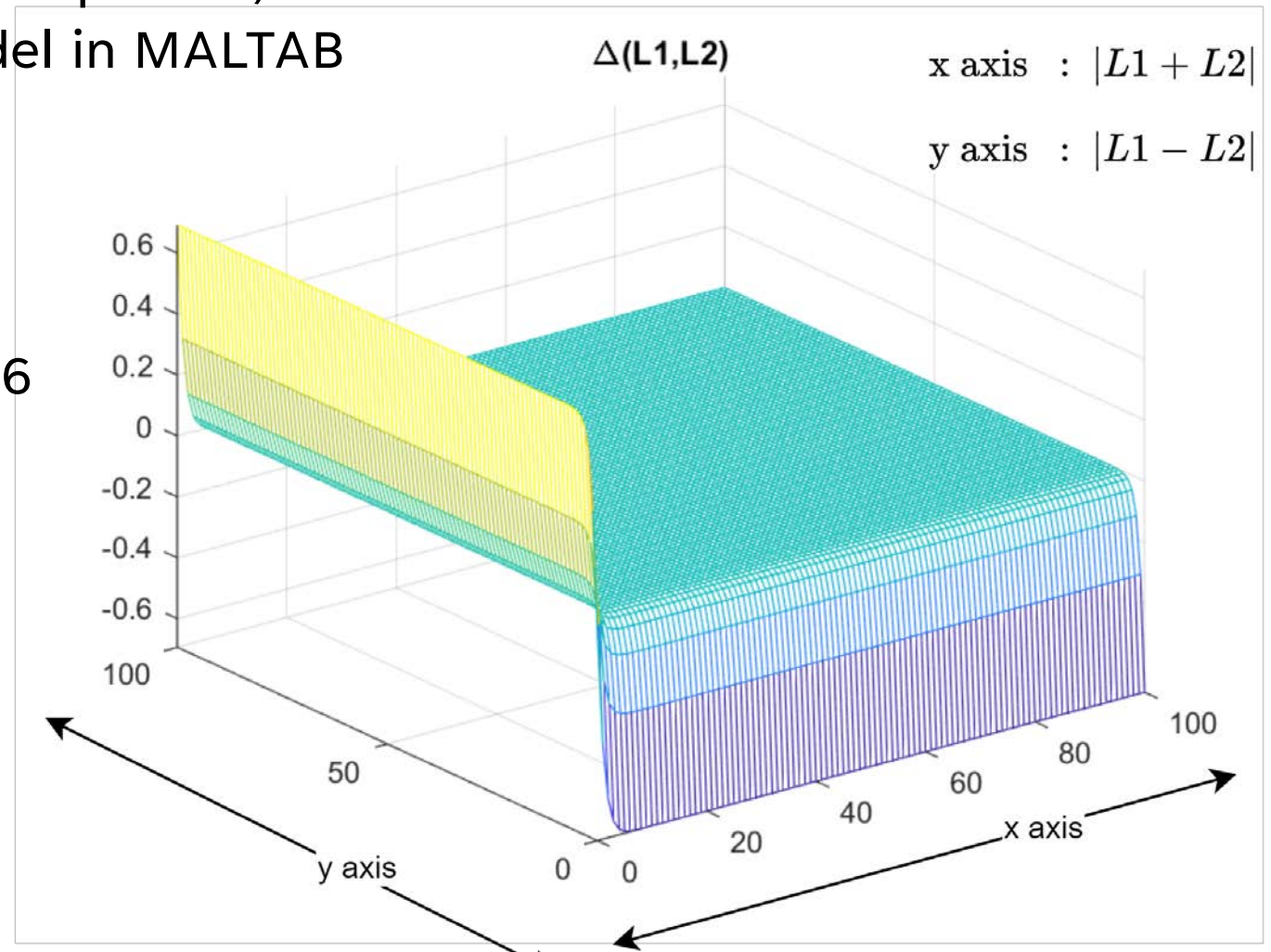
Problem

$$\text{CHK}(L_1, L_2) = \text{sgn}(L_1) \text{sgn}(L_2) \min(|L_1|, |L_2|) + \Delta(L_1, L_2).$$

- To solve the delta term in the CHK equation, simulating the mathematical model in MATLAB

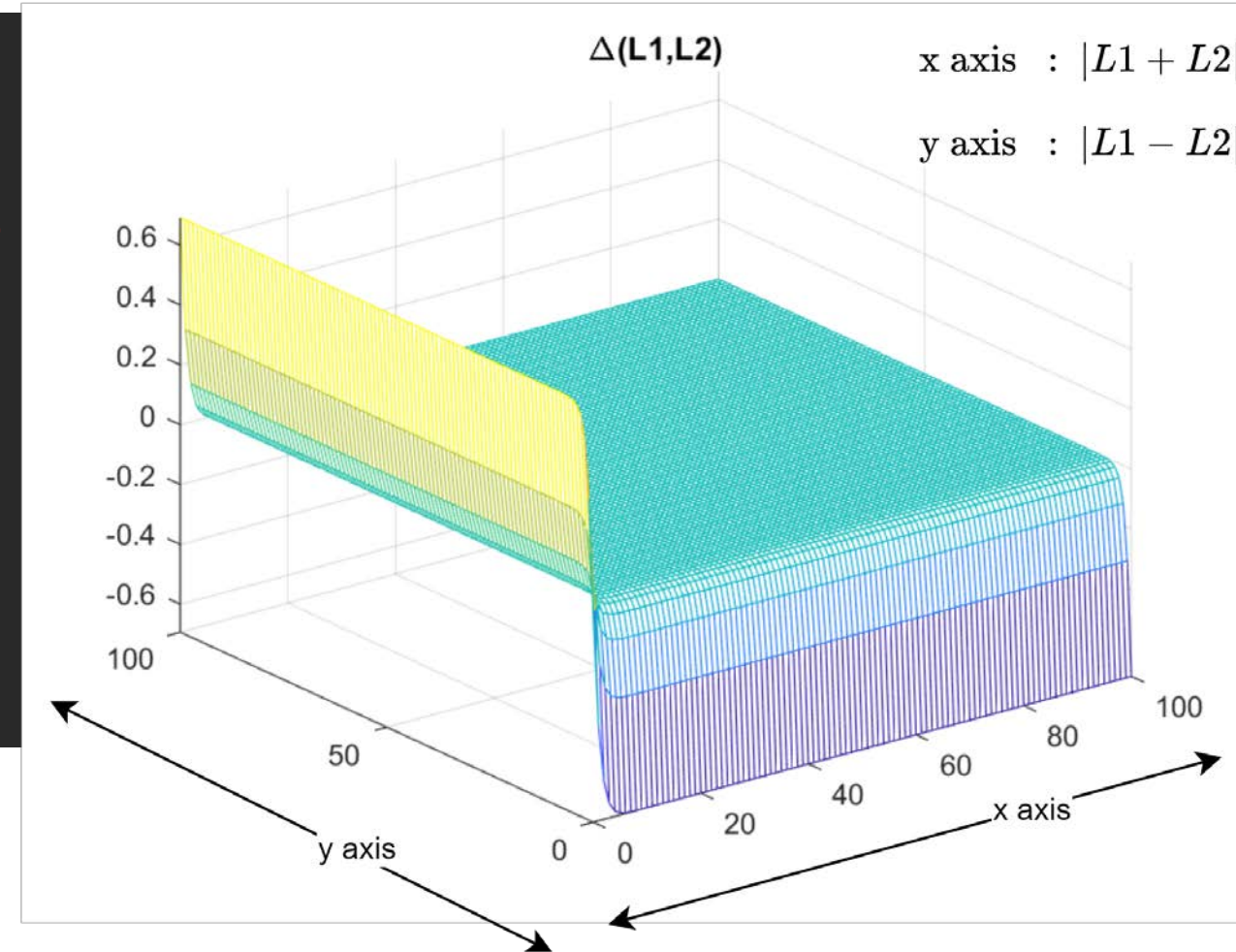
$$\Delta(L_1, L_2) = \ln \left(\frac{1 + e^{-|L_1 + L_2|}}{1 + e^{-|L_1 - L_2|}} \right)$$

- The maximum value is about 0.696



Problem

```
1  x = 0:100;  
2  y = 0:100;  
3  [xx, yy] = meshgrid(x, y);  
4  zzz = log((1+exp(-xx))./(1+exp(-yy)));  
5  
6  subplot(2,2,1); mesh(xx);  
7  title('xx'); axis tight  
8  subplot(2,2,2); mesh(yy);  
9  title('yy'); axis tight  
10 figure(2)  
11 subplot(1,1,1); mesh(xx, yy, zzz);  
12 title('\Delta(L1,L2)'); axis tight
```

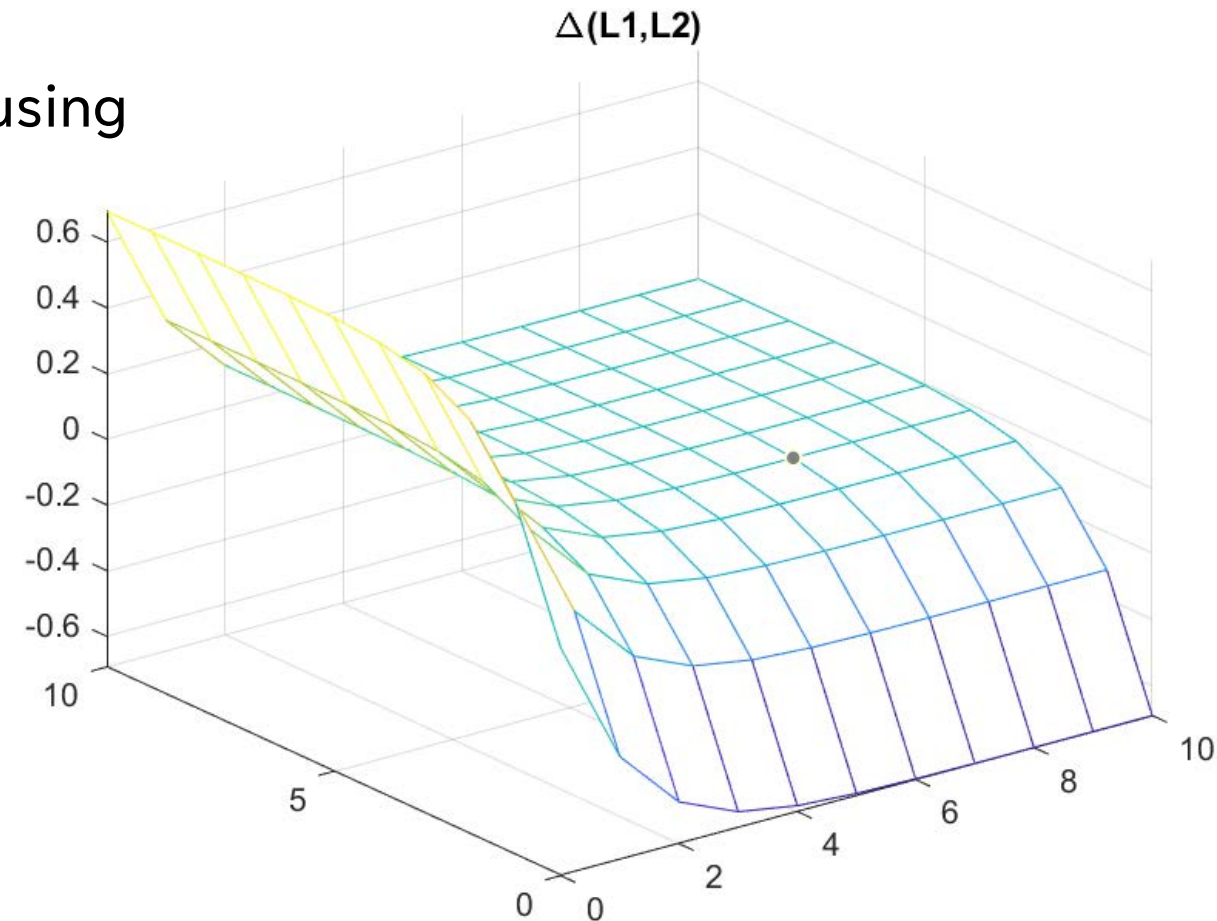


Problem

$$\text{CHK}(L_1, L_2) = \text{sgn}(L_1) \text{sgn}(L_2) \min(|L_1|, |L_2|) + \Delta(L_1, L_2).$$

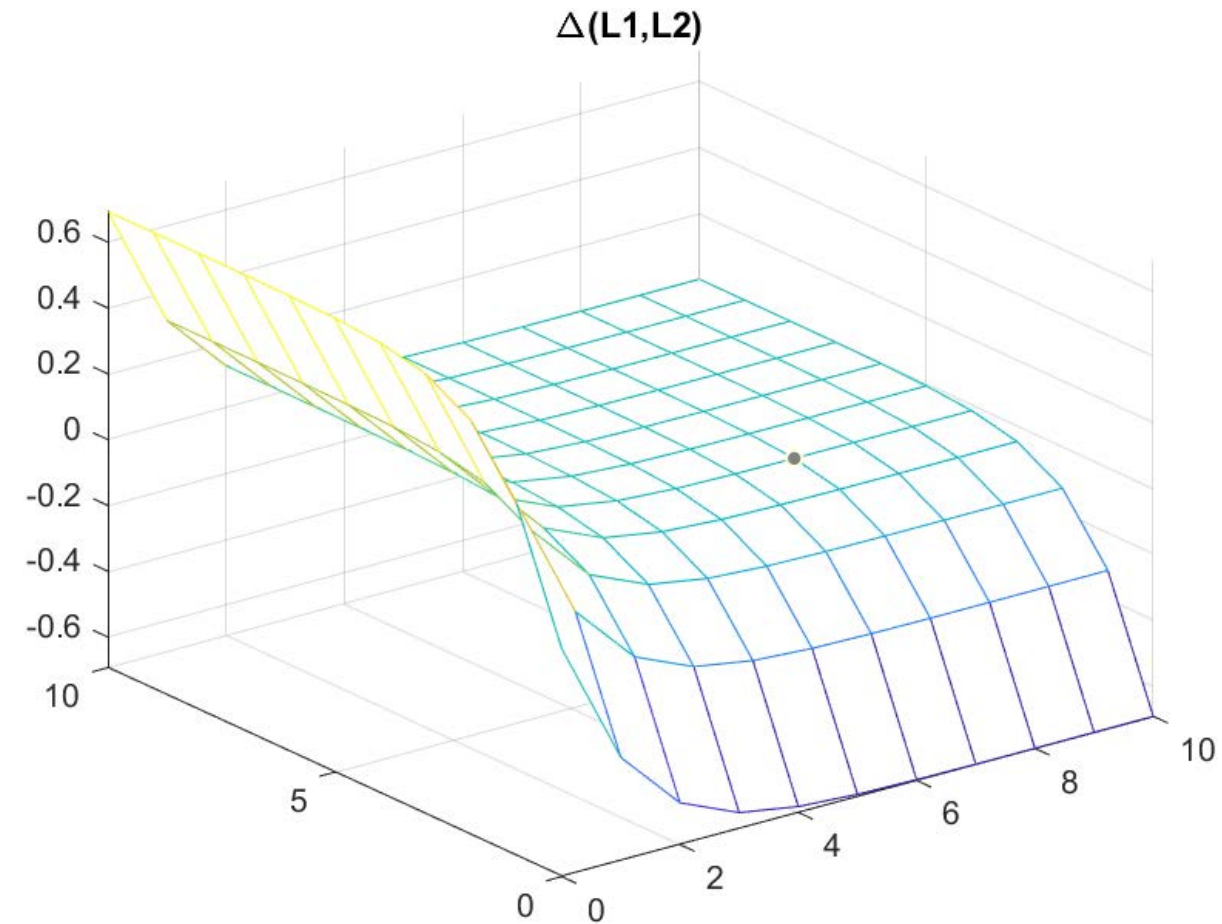
- The delta function will well converge when absolute value term is about 10.
- Build a 1000x1000 table for CHK function using
- Range in X:[0:0.01:10] , Y:[0:0.01:10]

$$\Delta(L_1, L_2) = \ln \left(\frac{1 + e^{-|L_1+L_2|}}{1 + e^{-|L_1-L_2|}} \right)$$



Problem

```
x = 0:10;  
y = 0:10;  
[xx, yy] = meshgrid(x, y);  
zzz = log((1+exp(-xx))./(1+exp(-yy)));  
  
subplot(2,2,1); mesh(xx);  
title('xx'); axis tight  
subplot(2,2,2); mesh(yy);  
title('yy'); axis tight  
figure(2)  
subplot(1,1,1); mesh(xx, yy, zzz);  
title('\Delta(L1,L2)'); axis tight
```



Problem

- MATLAB CODE

$$\text{CHK}(L_1, L_2) = \text{sgn}(L_1) \text{sgn}(L_2) \min(|L_1|, |L_2|) + \Delta(L_1, L_2).$$

$$\Delta(L_1, L_2) = \ln \left(\frac{1 + e^{-|L_1+L_2|}}{1 + e^{-|L_1-L_2|}} \right)$$

```
1 clear all ; close all ; clc
2 result=zeros(1000,1000,'double');
3 L1addL2 = 0:0.01:9.99;
4 L1minusL2 = 0:0.01:9.99;
5 for i = 1:1:1000
6     for j = 1:1:1000
7         result(i,j) = log((1+exp(-abs(L1addL2(j))) ) )/(1+exp(-abs(L1minusL2(i))) ) ) );
8     end
9 end
10 %1000x1000 matrix
11 fileID = fopen('deltaL1_L2.txt', 'w'); % Open the file for writing ('w' flag)
12 for row = 1:size(result, 1)
13     for col = 1:size(result, 2)
14         fprintf(fileID, '%.3f ', result(row, col)); % Write each element
15     end
16     fprintf(fileID, '\n'); % Start a new line after each row
17 end
18
19 fclose(fileID); % Close the file
```



Problem • deltaL1_L2.txt File

```
1 0.000 -0.005 -0.010 -0.015 -0.020 -0.025 -0.030 -0.034
2 0.005 0.000 -0.005 -0.010 -0.015 -0.020 -0.025 -0.029
3 0.010 0.005 0.000 -0.005 -0.010 -0.015 -0.020 -0.024
4 0.015 0.010 0.005 0.000 -0.005 -0.010 -0.015 -0.020
5 0.020 0.015 0.010 0.005 0.000 -0.005 -0.010 -0.015
6 0.025 0.020 0.015 0.010 0.005 0.000 -0.005 -0.010
7 0.030 0.025 0.020 0.015 0.010 0.005 0.000 -0.005
8 0.034 0.029 0.024 0.020 0.015 0.010 0.005 0.000
9 0.039 0.034 0.029 0.024 0.019 0.015 0.010 0.005
10 0.044 0.039 0.034 0.029 0.024 0.019 0.014 0.010
11 0.049 0.044 0.039 0.034 0.029 0.024 0.019 0.014
12 0.053 0.049 0.044 0.039 0.034 0.029 0.024 0.019
13 0.058 0.053 0.048 0.043 0.038 0.034 0.029 0.024
14 0.063 0.058 0.053 0.048 0.043 0.038 0.033 0.028
15 0.068 0.063 0.058 0.053 0.048 0.043 0.038 0.033
16 0.072 0.067 0.062 0.057 0.052 0.048 0.043 0.038
17 0.077 0.072 0.067 0.062 0.057 0.052 0.047 0.042
```

```
980 0.693 0.688 0.683 0.678 0.673 0.668 0.664 0.659 0.654
981 0.693 0.688 0.683 0.678 0.673 0.668 0.664 0.659 0.654
982 0.693 0.688 0.683 0.678 0.673 0.668 0.664 0.659 0.654
983 0.693 0.688 0.683 0.678 0.673 0.668 0.664 0.659 0.654
984 0.693 0.688 0.683 0.678 0.673 0.668 0.664 0.659 0.654
985 0.693 0.688 0.683 0.678 0.673 0.668 0.664 0.659 0.654
986 0.693 0.688 0.683 0.678 0.673 0.668 0.664 0.659 0.654
987 0.693 0.688 0.683 0.678 0.673 0.668 0.664 0.659 0.654
988 0.693 0.688 0.683 0.678 0.673 0.668 0.664 0.659 0.654
989 0.693 0.688 0.683 0.678 0.673 0.668 0.664 0.659 0.654
990 0.693 0.688 0.683 0.678 0.673 0.668 0.664 0.659 0.654
991 0.693 0.688 0.683 0.678 0.673 0.668 0.664 0.659 0.654
992 0.693 0.688 0.683 0.678 0.673 0.668 0.664 0.659 0.654
993 0.693 0.688 0.683 0.678 0.673 0.668 0.664 0.659 0.654
994 0.693 0.688 0.683 0.678 0.673 0.668 0.664 0.659 0.654
995 0.693 0.688 0.683 0.678 0.673 0.668 0.664 0.659 0.654
996 0.693 0.688 0.683 0.678 0.673 0.668 0.664 0.659 0.654
997 0.693 0.688 0.683 0.678 0.673 0.668 0.664 0.659 0.654
998 0.693 0.688 0.683 0.678 0.673 0.668 0.664 0.659 0.654
999 0.693 0.688 0.683 0.678 0.673 0.668 0.664 0.659 0.654
1000 0.693 0.688 0.683 0.678 0.673 0.668 0.664 0.659 0.654
```

```
-0.693 -0.693 -0.693 -0.693 -0.693 -0.693 -0.693
-0.688 -0.688 -0.688 -0.688 -0.688 -0.688 -0.688
0.683 -0.683 -0.683 -0.683 -0.683 -0.683 -0.683
0.678 -0.678 -0.678 -0.678 -0.678 -0.678 -0.678
0.673 -0.673 -0.673 -0.673 -0.673 -0.673 -0.673
0.668 -0.668 -0.668 -0.668 -0.668 -0.668 -0.668
0.664 -0.664 -0.664 -0.664 -0.664 -0.664 -0.664
-0.659 -0.659 -0.659 -0.659 -0.659 -0.659 -0.659
-0.654 -0.654 -0.654 -0.654 -0.654 -0.654 -0.654
0.649 -0.649 -0.649 -0.649 -0.649 -0.649 -0.649
0.644 -0.644 -0.644 -0.644 -0.644 -0.644 -0.644
0.640 -0.640 -0.640 -0.640 -0.640 -0.640 -0.640
0.635 -0.635 -0.635 -0.635 -0.635 -0.635 -0.635
0.630 -0.630 -0.630 -0.630 -0.630 -0.630 -0.630
-0.626 -0.626 -0.626 -0.626 -0.626 -0.626 -0.626
-0.621 -0.621 -0.621 -0.621 -0.621 -0.621 -0.621
0.616 -0.616 -0.616 -0.616 -0.616 -0.616 -0.616
0.612 -0.612 -0.612 -0.612 -0.612 -0.612 -0.612
0.607 -0.607 -0.607 -0.607 -0.607 -0.607 -0.607
```

```
0 -0.000 -0.000 -0.000 -0.000 -0.000 -0.000 -0.000 -0.000
-0.000 -0.000 -0.000 -0.000 -0.000 -0.000 -0.000 -0.000
-0.000 -0.000 -0.000 -0.000 -0.000 -0.000 -0.000 -0.000
0.000 -0.000 -0.000 -0.000 -0.000 -0.000 -0.000 -0.000
0.000 -0.000 -0.000 -0.000 -0.000 -0.000 -0.000 -0.000
0.000 -0.000 -0.000 -0.000 -0.000 -0.000 -0.000 -0.000
0.000 -0.000 -0.000 -0.000 -0.000 -0.000 -0.000 -0.000
0.000 -0.000 -0.000 -0.000 -0.000 -0.000 -0.000 -0.000
-0.000 -0.000 -0.000 -0.000 -0.000 -0.000 -0.000 -0.000
0.000 -0.000 -0.000 -0.000 -0.000 -0.000 -0.000 -0.000
0.000 -0.000 -0.000 -0.000 -0.000 -0.000 -0.000 -0.000
0.000 -0.000 -0.000 -0.000 -0.000 -0.000 -0.000 -0.000
0.000 -0.000 -0.000 -0.000 -0.000 -0.000 -0.000 -0.000
0.000 -0.000 -0.000 -0.000 -0.000 -0.000 -0.000 -0.000
0.000 -0.000 -0.000 -0.000 -0.000 -0.000 -0.000 -0.000
0.000 0.000 -0.000 -0.000 -0.000 -0.000 -0.000 -0.000
0.000 0.000 0.000 -0.000 -0.000 -0.000 -0.000 -0.000
0.000 0.000 0.000 0.000 -0.000 -0.000 -0.000 -0.000
0.000 0.000 0.000 0.000 0.000 -0.000 -0.000 -0.000
0.000 0.000 0.000 0.000 0.000 0.000 -0.000 -0.000
0.000 0.000 0.000 0.000 0.000 0.000 0.000 -0.000
```



Simulation

SPA Data and Figure

Simulation Data - SPA

- Error Block Bound = 50 , Max Iteration Bound = 50

SNR(dB)	0	0.2	0.4	0.6	0.8	1	1.2	1.4	1.6	1.8
Decoded Bit	39050	39050	39050	39050	39050	39050	39050	39050	39050	41393
Error Bit	4247	4107	3830	3726	3316	3192	3026	2888	2625	2475
BER	1.09E-01	1.05E-01	9.81E-02	9.54E-02	8.49E-02	8.17E-02	7.75E-02	7.40E-02	6.72E-02	5.98E-02
BLER	1.00E+00	1.00E+00	1.00E+00	1.00E+00	1.00E+00	1.00E+00	1.00E+00	1.00E+00	1.00E+00	9.43E-01
		2	2.2	2.4	2.6	2.8	3	3.2	3.4	3.6
		48422	52327	82786	207746	493592	1727572	7319532	29094593	1.36E+08
		2275	2210	2054	2073	2037	2049	2024	2030	2173
		4.70E-02	4.22E-02	2.48E-02	9.98E-03	4.13E-03	1.19E-03	2.77E-04	6.98E-05	1.60E-05
		8.06E-01	7.46E-01	4.72E-01	1.88E-01	7.91E-02	2.26E-02	5.34E-03	1.34E-03	2.87E-04

- Error Block Bound = 50 , Max Iteration Bound = 100

SNR(dB)	0	0.2	0.4	0.6	0.8	1	1.2	1.4	1.6	
Decoded Bit	3.91E+04	39050	39050	39050	39050	39050	39050	39050	39050	
Error Bit	4.26E+03	4106	3825	3723	3312	3195	3023	2893	2629	
BER	1.09E-01	1.05E-01	9.80E-02	9.53E-02	8.48E-02	8.18E-02	7.74E-02	7.41E-02	6.73E-02	
BLER	1.00E+00	1.00E+00	1.00E+00	1.00E+00	1.00E+00	1.00E+00	1.00E+00	1.00E+00	1.00E+00	
	1.8	2	2.2	2.4	2.6	2.8	3	3.2	3.4	3.6
	41393	47641	52327	97625	202279	478753	1403457	5358441	19209476	80249312
	2477	2262	2195	2096	2193	2076	2085	2131	2142	2113
	5.98E-02	4.75E-02	4.19E-02	2.15E-02	1.08E-02	4.34E-03	1.49E-03	3.98E-04	1.12E-04	2.63E-05
	9.43E-01	8.20E-01	7.46E-01	4.00E-01	1.93E-01	8.16E-02	2.78E-02	7.29E-03	2.03E-03	4.87E-04

Fixed Error Block
50,
Iteration times
50 100 150



Simulation Data - SPA

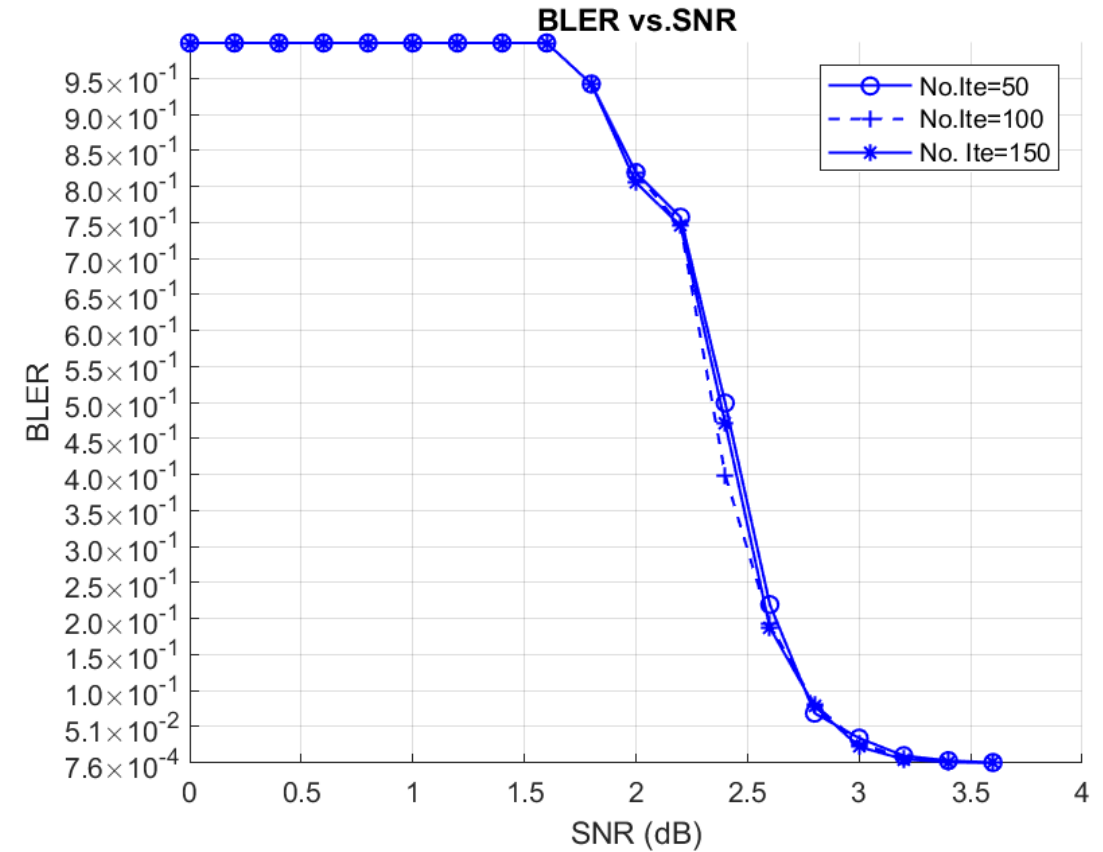
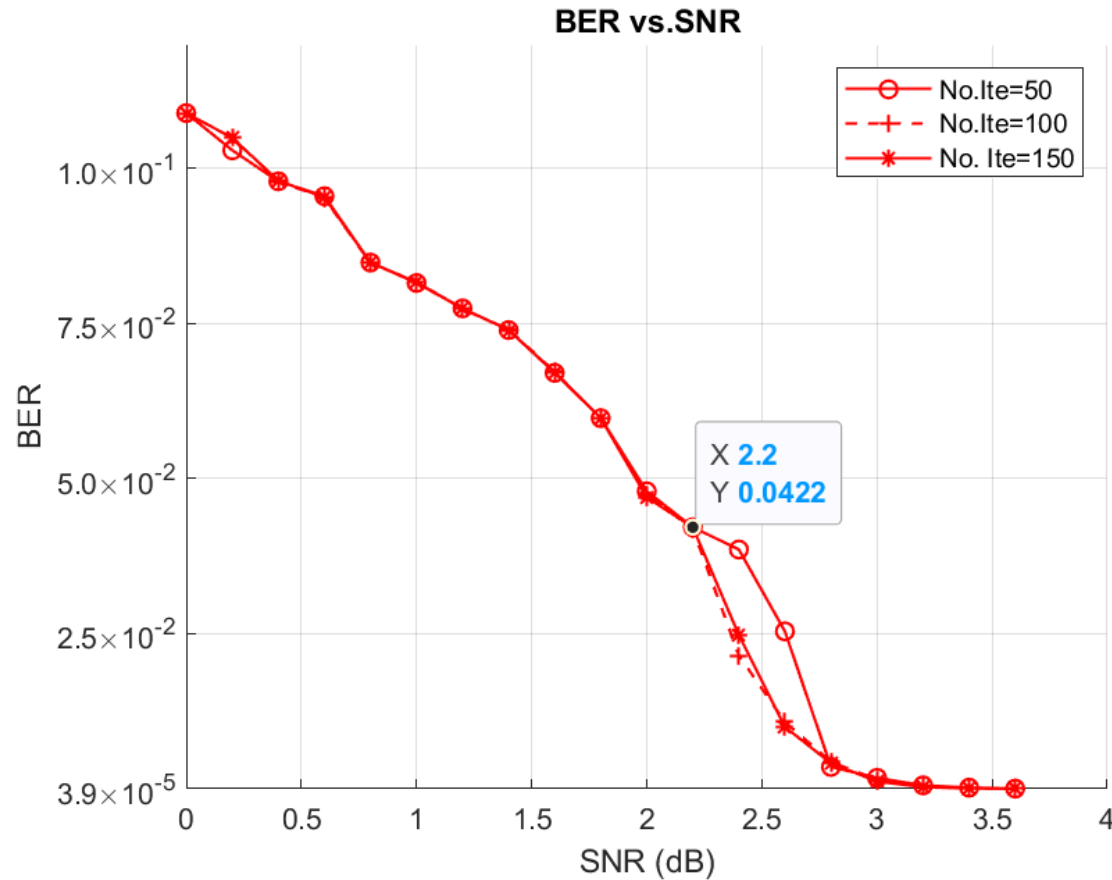
- Error Block Bound = 50 , Max Iteration Bound = 150

Fixed Error Block
50,
Iteration times
50 100 150

SNR(dB)	0	0.2	0.4	0.6	0.8	1	1.2	1.4	1.6	
Decoded Bit	39050	39050	39050	39050	39050	39050	39050	39050	39050	
Error Bit	4251	4109	3828	3732	3314	3188	3026	2888	2622	
BER	1.09E-01	1.03E-01	9.80E-02	9.56E-02	8.49E-02	8.16E-02	7.75E-02	7.40E-02	6.71E-02	
BLER	1.00E+00	1.00E+00	1.00E+00	1.00E+00	1.00E+00	1.00E+00	1.00E+00	1.00E+00	1.00E+00	
	1.8	2	2.2	2.4	2.6	2.8	3	3.2	3.4	3.6
	41393	47641	51546	78100	177287	562320	1130107	3830805	11677512	51117231
	2474	2287	2173	1990	1987	1993	1997	2140	2170	1984
	5.98E-02	4.80E-02	4.22E-02	3.86E-02	2.54E-02	3.54E-03	1.77E-03	5.59E-04	1.86E-04	3.88E-05
	9.43E-01	8.20E-01	7.58E-01	5.00E-01	2.20E-01	6.94E-02	3.46E-02	1.02E-02	3.34E-03	7.64E-04

Simulation Figure - SPA

- SNR超過2.2dB後，SPA算法在疊代50~100間的解碼率有顯著提升，超過100並無顯著增加系統效能，故後續模擬將取疊代70次為基準





Simulation

MSA Data and Figure

Simulation Data - MSA

- Error Block Bound = 50 , Max Iteration Bound = 70

Iteration Bound
100
Different Error Block
50 100 200

SNR(dB)	0	0.2	0.4	0.6	0.8	1	1.2	1.4	1.6	1.8	2	2.2
Decoded Bit	39050	39050	39050	39050	39050	39050	39050	39050	39050	39050	39050	39050
Error Bit	5931	5744	5461	5349	5107	4902	4898	4767	4644	4619	4434	4285
BER	1.52E-01	1.47E-01	1.40E-01	1.37E-01	1.31E-01	1.26E-01	1.25E-01	1.22E-01	1.19E-01	1.18E-01	1.14E-01	1.10E-01
BLER	1.00E+00	1.00E+00	1.00E+00	1.00E+00	1.00E+00	1.00E+00	1.00E+00	1.00E+00	1.00E+00	1.00E+00	1.00E+00	1.00E+00
time cost	67.641	67.69	68.206	66.639	81.684	83.054	88.217	97.082	96.353	89.192	93.52	91.584
	2.4	2.6	2.8	3	3.2	3.4	3.6	3.8	4	4.2	4.4	4.6
	39050	39050	39831	44517	56232	78100	149171	302247	877844	4737546	22364716	219606266
	4.26E+03	3919	3909	3814	4037	3985	3855	3747	3551	3619	3680	3447
	1.09E-01	1.00E-01	9.81E-02	8.57E-02	7.18E-02	5.10E-02	2.58E-02	1.24E-02	4.05E-03	7.64E-04	1.65E-04	1.57E-05
	1.00E+00	1.00E+00	9.80E-01	8.77E-01	6.94E-01	5.00E-01	2.62E-01	1.29E-01	4.45E-02	8.24E-03	1.75E-03	1.78E-04
	98.184	89.654	86.964	121.781	95.259	101.491	65.486	71.115	94.861	230.396	747.314	26396.956

Simulation Data - MSA

- Error Block Bound = 100 , Max Iteration Bound = 70

Iteration Bound

100

Different Error Block

50 100 200

SNR(dB)	0	0.2	0.4	0.6	0.8	1	1.2	1.4	1.6	1.8	2	2.2
Decoded I	78100	78100	78100	78100	78100	78100	78100	78100	78100	78100	78100	78100
Error Bit	11719	11268	10850	10700	10062	9984	9692	9513	9389	9220	8685	8626
BER	1.50E-01	1.44E-01	1.39E-01	1.37E-01	1.29E-01	1.28E-01	1.24E-01	1.22E-01	1.20E-01	1.18E-01	1.11E-01	1.10E-01
BLER	1.00E+00	1.00E+00	1.00E+00	1.00E+00	1.00E+00	1.00E+00	1.00E+00	1.00E+00	1.00E+00	1.00E+00	1.00E+00	1.00E+00
time cost	150.931	135.576	147.679	170.9	191.227	184.569	220.531	234.144	243.14	588.572	237.893	217.676
	2.4	2.6	2.8	3	3.2	3.4	3.6	3.8	4	4.2	4.4	4.6
	78100	78100	81224	89815	116369	145266	276474	598246	1857218	7862327	45929829	415981687
	8535	8168	8089	8130	7875	7835	7610	7342	7344	7665	6953	7338
	1.09E-01	1.05E-01	9.96E-02	9.05E-02	6.77E-02	5.39E-02	2.75E-02	1.23E-02	3.95E-03	9.75E-04	1.51E-04	1.76E-05
	1.00E+00	1.00E+00	9.62E-01	8.70E-01	6.71E-01	5.38E-01	2.82E-01	1.31E-01	4.21E-02	9.93E-03	1.70E-03	1.88E-04
	128.443	129.179	131.935	127.825	122.683	126.41	133.461	146.143	197.639	402.546	1558.808	32013.996

Simulation Data - MSA

- Error Block Bound = 200 , Max Iteration Bound = 70

Iteration Bound

100

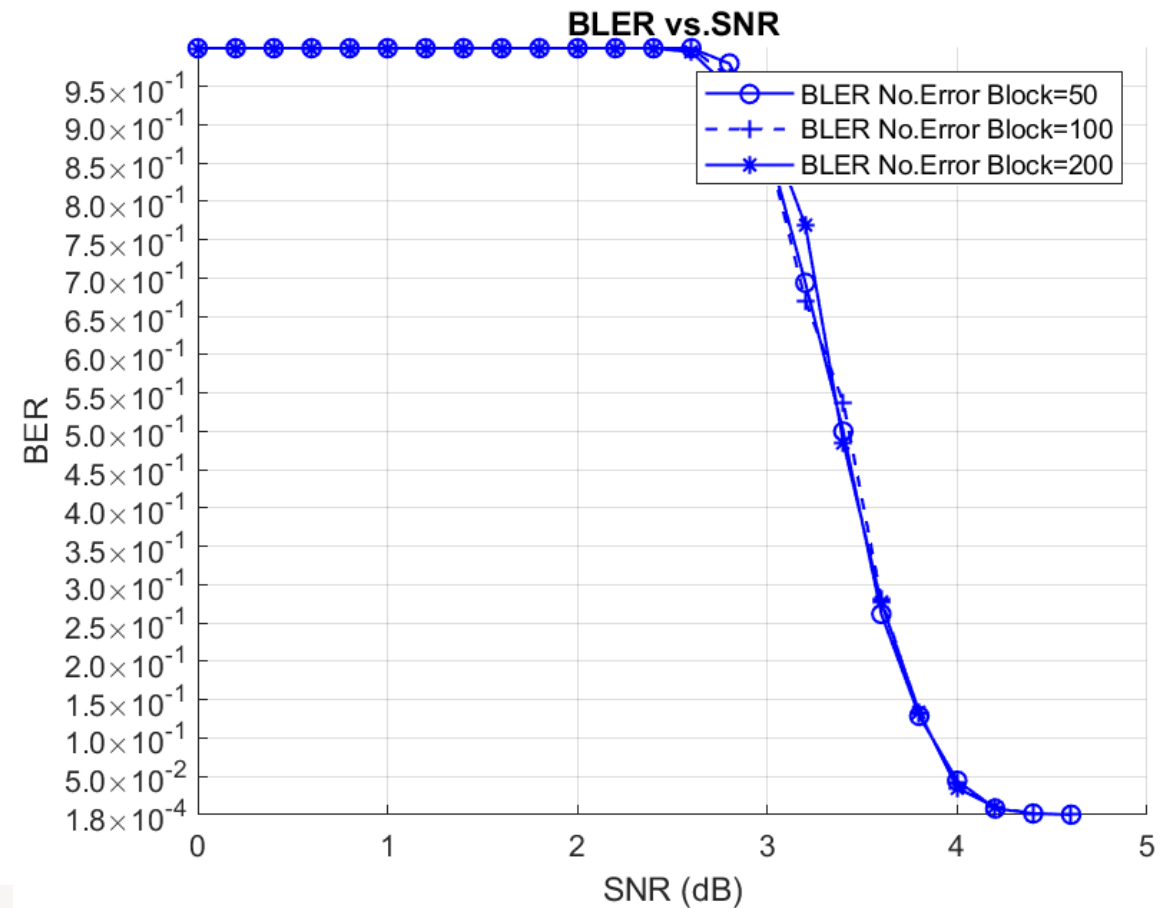
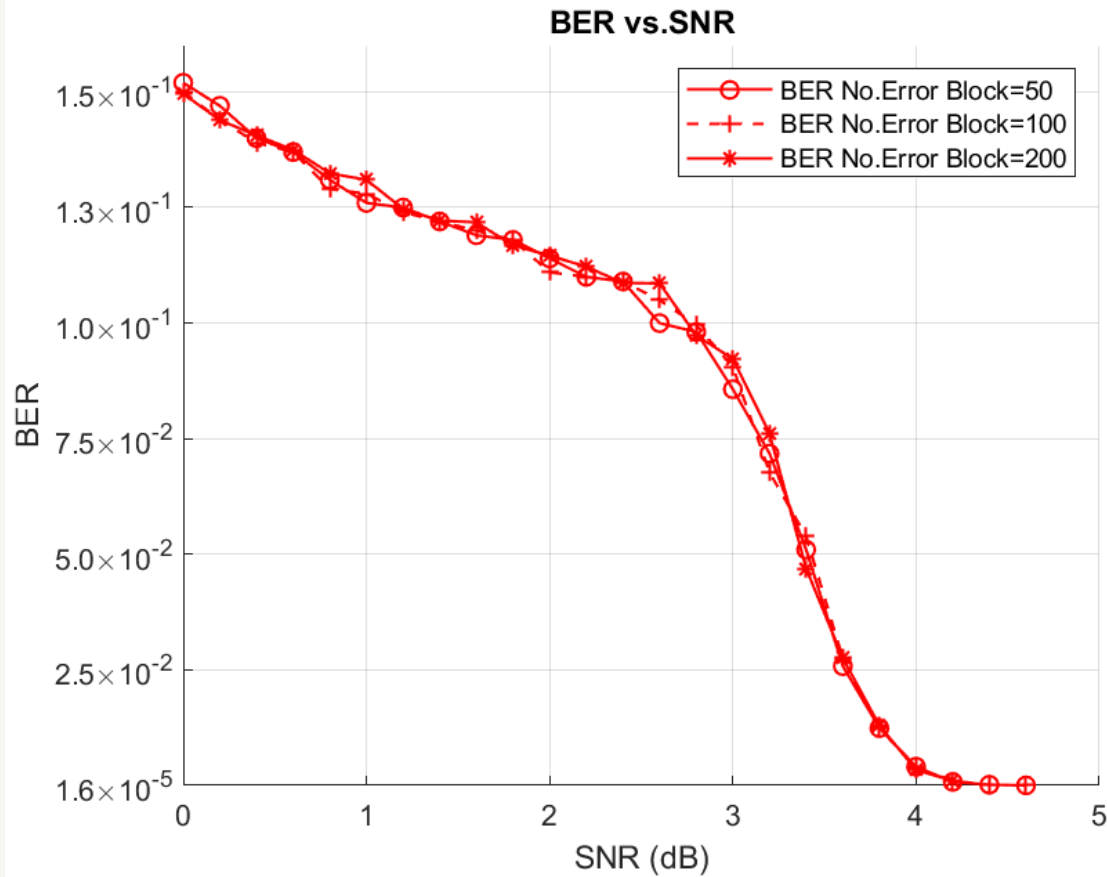
Different Error Block

50 100 200

SNR(dB)	0	0.2	0.4	0.6	0.8	1	1.2	1.4	1.6	1.8	2	
Decoded I	156200	156200	156200	156200	156200	156200	156200	156200	156200	156200	156200	
Error Bit	23361	22472	21960	21474	20676	20470	19479	19087	19026	18245	17901	
BER	1.496E-01	1.439E-01	1.406E-01	1.375E-01	1.324E-01	1.310E-01	1.247E-01	1.222E-01	1.218E-01	1.168E-01	1.146E-01	
BLER	1.00E+00	1.00E+00	1.00E+00	1.00E+00	1.00E+00	1.00E+00	1.00E+00	1.00E+00	1.00E+00	1.00E+00	1.00E+00	
time cost	283	259	244	242	318	303	308	322	317	321	317	
	2.2	2.4	2.6	2.8	3	3.2	3.4	3.6	3.8	4	4.2	4.4
	156200	156200	156981	164791	173382	203060	321772	563882	1161347	4457167	15315410	88042130
	17519	16981	17054	16040	15996	15460	15094	15539	15022	14924	15251	15063
	1.122E-01	1.087E-01	1.086E-01	9.734E-02	9.226E-02	7.614E-02	4.691E-02	2.756E-02	1.293E-02	3.348E-03	9.958E-04	1.71E-04
	1.00E+00	1.00E+00	9.95E-01	9.48E-01	9.01E-01	7.69E-01	4.85E-01	2.77E-01	1.34E-01	3.50E-02	1.02E-02	1.77E-03
	276.9	301	310	298	254	289.9	323	345.56	359	535.25	1028	33218

Simulation Figure - MSA

- Block超過一定數量後，也就是錯的bit數超過一定數量，系統穩定沒有太大改變





Simulation

SPA and MSA



Simulation Compare

- Comparing arguments

Max Error Block = 100 , Iteration Bound = 70

- SPA

SNR(dB)	0	0.2	0.4	0.6	0.8	1	1.2	1.4	1.6
Decoded Bit	78100	78100	78100	78100	78100	78100	78100	78100	78881
Error Bit	8519	8098	7546	7234	6794	6495	5970	5644	5166
BER	1.09E-01	1.04E-01	9.66E-02	9.26E-02	8.70E-02	8.32E-02	7.64E-02	7.23E-02	6.55E-02
BLER	1.00E+00	1.00E+00	1.00E+00	1.00E+00	1.00E+00	1.00E+00	1.00E+00	1.00E+00	9.90E-01
time cost	169.165	209.92	206.871	207.935	211	210	206	208	190
	1.8	2	2.2	2.4	2.6	2.8	3	3.2	3.4
	78881	90596	113245	178849	409244	1022329	2586672	8812023	37026429
	4904	4517	4382	4181	4108	4079	4073	4283	4278
	6.22E-02	4.99E-02	3.87E-02	2.34E-02	1.00E-02	3.99E-03	1.57E-03	4.86E-04	1.16E-04
	9.90E-01	8.62E-01	6.90E-01	4.37E-01	1.91E-01	7.64E-02	3.02E-02	8.86E-03	2.11E-03
	215	238	245	273	321	473	681	3049	6001

Simulation Compare

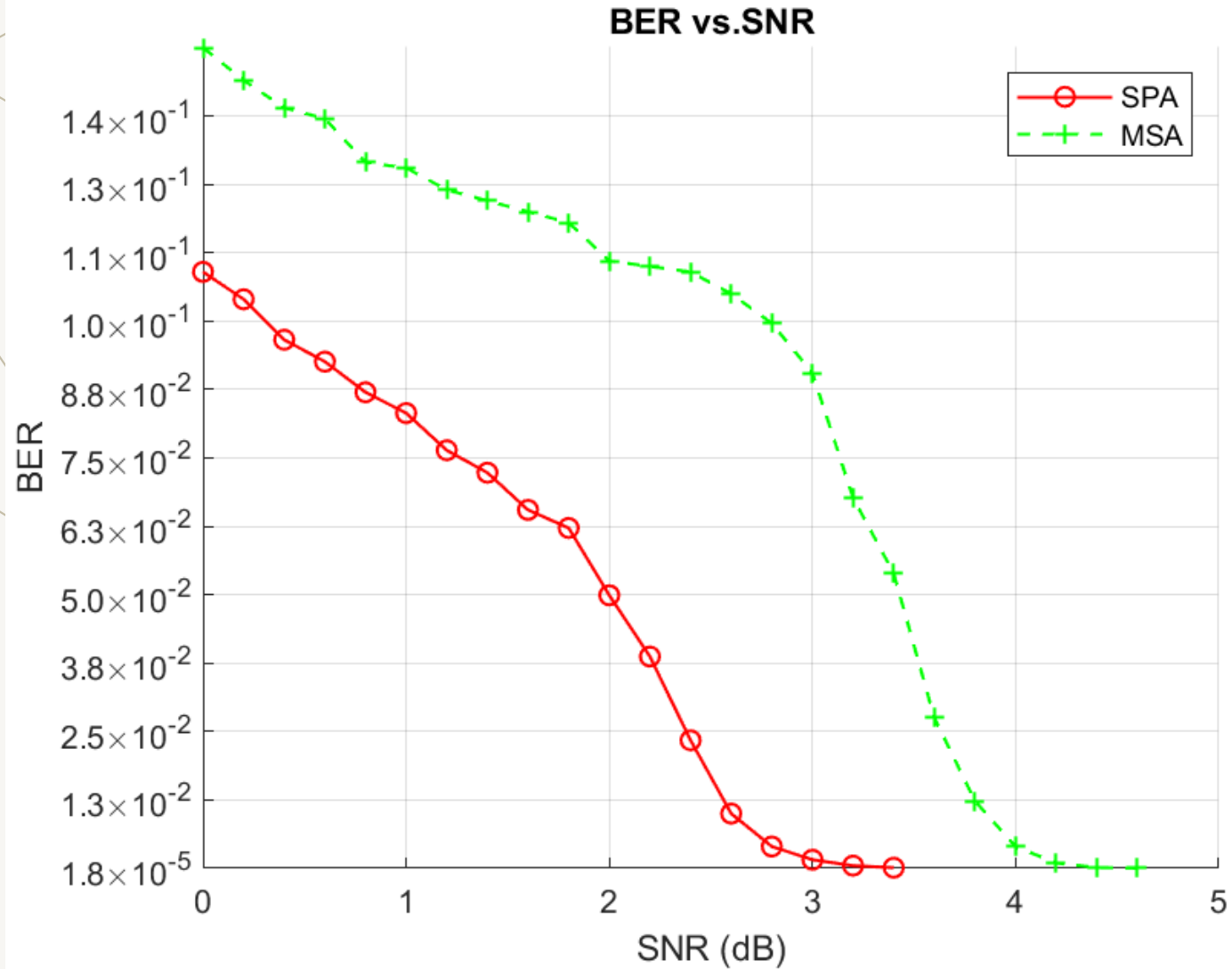
- Comparing arguments
Max Error Block = 100 , Iteration Bound = 70

- MSA

SNR(dB)	0	0.2	0.4	0.6	0.8	1	1.2	1.4	1.6	1.8	2	2.2
Decoded I	78100	78100	78100	78100	78100	78100	78100	78100	78100	78100	78100	78100
Error Bit	11719	11268	10850	10700	10062	9984	9692	9513	9389	9220	8685	8626
BER	1.50E-01	1.44E-01	1.39E-01	1.37E-01	1.29E-01	1.28E-01	1.24E-01	1.22E-01	1.20E-01	1.18E-01	1.11E-01	1.10E-01
BLER	1.00E+00	1.00E+00	1.00E+00	1.00E+00	1.00E+00	1.00E+00	1.00E+00	1.00E+00	1.00E+00	1.00E+00	1.00E+00	1.00E+00
time cost	150.931	135.576	147.679	170.9	191.227	184.569	220.531	234.144	243.14	588.572	237.893	217.676
	2.4	2.6	2.8	3	3.2	3.4	3.6	3.8	4	4.2	4.4	4.6
	78100	78100	81224	89815	116369	145266	276474	598246	1857218	7862327	45929829	415981687
	8535	8168	8089	8130	7875	7835	7610	7342	7344	7665	6953	7338
	1.09E-01	1.05E-01	9.96E-02	9.05E-02	6.77E-02	5.39E-02	2.75E-02	1.23E-02	3.95E-03	9.75E-04	1.51E-04	1.76E-05
	1.00E+00	1.00E+00	9.62E-01	8.70E-01	6.71E-01	5.38E-01	2.82E-01	1.31E-01	4.21E-02	9.93E-03	1.70E-03	1.88E-04
	128.443	129.179	131.935	127.825	122.683	126.41	133.461	146.143	197.639	402.546	1558.808	32013.996

Simulation Figure

- SPA V.S. MSA





Thanks

- 參考資料 老師講義
- 感謝 BoBo Lin的電腦: I5 11代 3060RTX